Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Ders kitabının 8. bölümü: web taraması (web crawling).
- Tarayıcı (crawler) neden gerekir, nasıl çalışır.
- Durdurma koşulları (stopping condition).
- `robots.txt` dosyası ve tarayıcı kuralları.
- Tarayıcıların kullanım alanları.
- Tarayıcı sınıflandırması (taxonomy): evrensel (universal) ve tercihli (preferential) tarayıcılar.
- Odaklı (focused) ve konusal (topical) tarayıcılar.
- Sıralı (sequential) temel tarayıcının modülleri.
- Genişlik öncelikli (breadth-first) ve derinlik öncelikli (depth-first) tarama.
- Uygulama sorunları: tekrar indirme, sınır listesi (frontier) büyümesi, indirici (fetcher) sağlamlığı, ayrıştırma (parsing), örümcek tuzakları (spider trap), URL düzgünleştirme (canonicalization).
- Sayfa deposu (page repository) seçenekleri.
- Eş zamanlı (concurrent) tarayıcılar.
- Evrensel tarayıcı mimarisi; kapsam (coverage) ile güncellik (freshness) dengesi.
- Tercihli tarama algoritmaları: Best-First (büyük olasılıkla Best-N-First), PageRank tabanlı, Fish Search, Shark Search, InfoSpider.
- Tarama boyutu–duyarlılık (crawl size–recall) grafikleriyle karşılaştırma.

Hocanın Özellikle Vurguladığı Kısımlar

- Proje bilgileri ve vize notları dersin sayfasında yayınlandı. Hoca bazı projelere yorum yazdı ve ek isteklerini sayfaya not etti.
- Projesini gönderdiği hâlde hocaya ulaşmayan öğrenci haber vermeli. Gönderdiğini gösteren e-postanın ekran görüntüsünü almalı. Aksi hâlde proje notu olmaz.
- Arama ham sayfalar üzerinde yapılmaz. Arama ters dizin (inverted index) üzerinde yapılır. Tarayıcı bu dizin için sayfaları toplar.
- Tohum URL (seed URL) seçimi çok önemli. Yanlış tohumla başlayan tarayıcı web'in ilgisiz yerlerini tarar. İstenen sayfalara çok geç ulaşır.
- İndirme modülü en çok zaman alan modüldür. Sebep: ağ gecikmesi (network latency).
- Universal ve focused tarayıcılar çevrim dışı (offline) çalışır, arama anında çalışmaz. Topical tarayıcılar arama anında (gerçek zamanlı) çalışır.
- Eş zamanlı tarayıcıda iş parçacıkları (thread) sınır listesini ve depoyu paylaşır. Bu paylaşılan verilere erişim senkronize edilmeli.
- Evrensel tarayıcıda kapsam ile güncellik arasında bir denge (trade-off) var. Tarayıcı bu dengeyi yapılandırılabilir biçimde desteklemeli.
- Tercihli tarayıcıları karşılaştırırken bakılacak yer: düşük tarama boyutunda duyarlılığı yüksek olan algoritma iyidir.

Kısa Tekrar Notları

- Tarayıcı: farklı sunuculardaki HTML sayfalarını bir yönteme göre bulan, indiren ve merkezi bir ortamda saklayan uygulama.
- Diğer adları: spider, robot, web agent. Örnek: Googlebot, MSNbot.

- Döngü: tohum URL'ler, HTTP isteği, sayfayı al, içinden yeni URL çıkar, sınır listesine ekle, tekrar et.
- Durdurma koşulları:
  - Süre (ağ gecikmesi yüzünden güvenilir değil).
  - İndirilen belge sayısı.
  - Toplam veri boyutu (ör. "şu kadar terabayt").
  - Ağaç derinliği (tohumdan itibaren ör. 6 seviye).
- `robots.txt`: sitede hangi sayfaların taranabileceğini söyler. Hocaya göre kurala uymayan tarayıcı güvenlik duvarınca işaretlenir ve o siteyi bir daha tarayamaz. Bu büyük olasılıkla her sitede otomatik gerçekleşen bir sonuç değildir.
- Breadth-first: sınır listesi FIFO kuyruk. Seviye seviye tarar.
- Depth-first: sınır listesi LIFO yığın. Bir dalı en dibe kadar tarar, sonra başka dala geçer.
- Tekrar indirmeyi önlemek için daha önce ziyaret edilen URL'ler bir hash tablosunda tutulur.
- Göreli URL (relative URL), temel URL (base URL) ile birleştirilip mutlak URL'ye (absolute URL) çevrilir.
- Spider trap: siteyi sürekli yeni dinamik URL'lerle tarayıcıyı içinde dolaştırır. Önlem: URL uzunluğu sınırı, aynı siteden taranan URL sayısı sınırı.
- Universal tarayıcı iyileştirmeleri: DNS önbelleği (cache), eşzamansız soket (asynchronous socket), yazılım tabanlı ağ ve depolama anahtarları (switch).
- Tercihli tarama: her URL'ye önem puanı verilir. Sınır listesi bu puana göre sıralanır.

Detaylı Açıklamalar

Tarayıcı neden gerekir

- Arama motorunda arama ters dizin tablosu üzerinde yapılır.
- Bu tabloyu kurmak için önce web'deki sayfalar toplanıp merkezi bir depoda saklanmalı.
- Sonra depo üzerinde metin ve bağlantı analizi yapılır. Ters dizin oluşturulur. Her URL için bir sıralama (ranking) değeri hesaplanır.
- Hoca bir örnek gösterir: bir sayfayı arama sonucunda gösterebilmek için tohumdan başlayıp en az 3 seviye taramak gerekmiş.
- Sonuç: iyi arama sonucu için olabildiğince çok ve ilgili (relevant) sayfa dizinlenmeli. Ör. eğlence sayfaları isteniyorsa tohum URL'ler en iyi eğlence sayfalarına götürmeli.
- Tarayıcılar internet bant genişliğini (bandwidth) en çok kullanan uygulamalardır.

Kullanım alanları

- Genel arama motorları için dizin kurmak.
- Dikey (vertical) arama motorları: yemek tarifi, haber (Google News gibi), kullanıcı yorumları.
- İş zekâsı (business intelligence): rakip firmaların ürünlerini ve haberlerini izleyip günlük rapor üretmek. Hoca bir gıda firmasının yeni ürün örneğini verir.
- Site izleme: birden fazla e-ticaret sitesinden fiyat, özellik, kampanya ve kargo bilgisini toplayan karma (hybrid) arama motoru. Hoca bazı öğrencilerin projesinde buna benzer bir şey olduğunu söyler.
- Kötü kullanım örneği: istenmeyen e-posta (spam) ve oltalama (phishing) için e-posta adresi toplayan tarayıcılar.

Tarayıcı sınıflandırması

```
Tarayıcılar
├── Universal (genel arama motorları: Googlebot, MSNbot)
└── Preferential (belirli amaca yönelik)
    ├── Focused (offline; ters dizin kurar; ör. yemek tarifi, online alışveriş)
    └── Topical (arama anında çalışır)
        ├── Static (önceden belirli dolaşma: BFS, DFS, best-first, PageRank)
        └── Adaptive (dolaşırken öğrenir: reinforcement learning, evolutionary)
```

- Topical tarayıcı: kullanıcı sorguyu girince tarama başlar. Ör. “web’de benim adım hangi sayfalarda geçiyor”.
- Sonuçlar bulundukça ekrana akar. Arayüz ya akan sonuç listesi ya da bildirim (notification) tabanlıdır. Ör. sonuçlar belli aralıklarla e-postaya gönderilir.
- Adaptive tarayıcı: indirilen sayfanın anahtar kelimelerini başlangıç kelimeleriyle karşılaştırır. Benzerlik yüksekse doğru zinciri takip ettiğini anlar. Yeni kelimeleri sorguya ekler. Sonraki URL’yi bu zenginleştirilmiş kelime grubuyla seçer. Hoca buna pekiştirmeli öğrenme (reinforcement learning) der.

Sıralı temel tarayıcı (basic sequential crawler)

- Paralellik yok. Tek iş parçacığında çalışan mantıksal akış.

```
Tohum URL'ler
  -> 1. Sınır listesini başlat (frontier; bağlı liste, bellekte)
  -> 2. Listenin başından URL al
  -> 3. HTTP isteği gönder, sayfayı al (en yavaş adım)
  -> 4. Sayfadan URL'leri çıkar, sınır listesine ekle
  -> 5. Sayfayı depoya kaydet
  -> 6. Durdurma koşulu sağlandı mı? Hayır: 2'ye dön
```

- URL’ler sınır listesine ayrıştırıcının (parser) görme sırasıyla eklenir. Ama sıra popülerliğe göre de kurulabilir.
- Depoya tüm HTML yazılmaz. Resim ve çoklu ortam nesneleri gereksizdir. Seçenekler: anahtar kelime vektörü olarak saklamak veya yalnız metni saklamak.
- Hoca Perl ile yazılmış örnek kodu anlatır:
  - Tohum URL’ler dosyadan okunur, sınır listesine konur.
  - Döngü taranan bağlantı sayısı en büyük değeri geçmeyene kadar sürer.
  - Listenin başındaki URL alınır, sayfa indirilir, dizinlenir.
  - Sayfadaki bağlantılar çıkarılır. Sayfanın kendisine işaret eden bağlantı atılır.
  - Bağlantılar düzgün biçime (canonical form) çevrilip sınır listesine eklenir.

BFS ve DFS

- Breadth-first search:
  - Sınır listesi FIFO.
  - Her sayfadan çıkan URL’ler listenin sonuna eklenir. Listenin başından alınır.
  - Web’i seviye seviye tarar.
- Depth-first search:
  - LIFO mantığı.
  - Sayfadaki ilk URL’ye gidilir, oradan çıkan ilk URL’ye gidilir. Durdurma koşulundaki derinliğe ulaşınca bir adım geri dönülür ve aynı sayfadaki sonraki URL alınır.
  - Belirli bir konuyu dibine kadar taramak için kullanılabilir.
- Bunlar statik çizge dolaşma (graph traversal) algoritmalarıdır. Tarayıcının yapılandırma dosyasında seçilebilir.

Uygulama sorunları

- Aynı sayfayı iki kez indirmemek:
  - Sınır listesinden çıkan URL’ler artık listede değildir ama daha önce indirilmiştir.
  - Ziyaret edilen tüm sayfalar bir hash tablosunda tutulur. Tabloda olan URL tekrar eklenmez.
- Sınır listesinin hızlı büyümesi:
  - Liste büyüyünce tarayıcı en önemli sayfalara geç ulaşır.
  - Liste bir mantığa göre (ör. popülerlik) sıralanmalı.
- İndirici (fetcher) sağlamlığı:
  - Bant genişliği düşük sunucularda indirme çok uzayabilir.
  - Zaman aşımı (timeout) konur: süre dolunca o URL bırakılır, sonrakine geçilir.
  - Çok büyük sayfanın yalnız bir kısmı indirilir veya sayfa atlanır.
  - URL’ler PDF, Word, ikili (binary) dosya gibi şeylere de işaret eder. Yalnız ayrıştırılabilen ve anahtar kelime çıkarılabilen dosyalar indirilmeli.

- Ayrıştırma (parsing):
  - Tarayıcılar (browser) eksik HTML'i de doğru gösterir. Tarayıcı (crawler) kodu da eksik ve hatalı HTML'e dayanıklı yazılmalı.
- Durak kelimeler (stop words): kendi başına anlamı olmayan kelimeler saklanmadan önce atılır.
- Kök bulma (stemming): aynı kökten kelimeler tek köke çevrilir. Dizin küçülür.
- Statik ve dinamik URL:
  - Dinamik URL'ler (ör. CGI betikleri) örümcek tuzağı kurabilir.
  - Tuzakta site sürekli yeni dinamik URL üretip tarayıcıyı aynı sitede dolaştırır.
  - Önlem 1: URL uzunluğu belli bir karakter sayısını aşınca sınır listesine eklenmez (spam URL olabilir).
  - Önlem 2: aynı alan adından (domain) çok sayıda URL taranmışsa o sitenin yeni dinamik URL'leri eklenmez.
- Göreli URL: ör. sayfada yalnız `intro.html` geçebilir. Temel URL ile birleştirilip mutlak URL yapılır.
- Düzgünleştirme (canonicalization):
  - Hoca aynı sayfaya işaret eden 4 farklı URL gösterir.
  - Farklar: küçük ve büyük harf, standart portun yazılması, önce alt dizine girip sonra köke dönen yol parçası.
  - Hepsi tek standart biçime çevrilir. Tarayıcı bunların aynı URL olduğunu anlamalı.

Sayfa deposu (page repository)

- Çok sayıda sayfa varsa sayfalar sıkıştırılarak veya yalnız gerekli metin tutularak saklanır.
- Seçenekler:
  - Dosya sistemi: URL bir hash fonksiyonuyla dosya adına çevrilir. Her sayfa ayrı dosya olur. Çok sayıda dosya oluşur.
  - Büyük tek bir XML dosyası: her eleman bir sayfa.
  - İlişkisel veritabanı.
  - Bu iş için özel, yükü (overhead) düşük veritabanları. Hoca bir örnek adı söyler; altyazıda "birthday DB" diye geçer (büyük olasılıkla Berkeley DB).

Eş zamanlı tarayıcı (concurrent crawler)

- Günümüzde hiçbir gerçek tarayıcı sıralı çalışmaz.
- Sıralı tarayıcıda zaman alan adımlar:
  - DNS sorgusu: URL'deki ana makine adını (hostname) IP adresine çevirmek.
  - Senkron soket: istek gönderilince yanıt gelene kadar soket kilitlenir (block).
  - Ağ süresi: istekten yanıta kadar geçen süre.
- Hoca soketi hatırlatır: TCP/IP ile iki uygulama konuşurken kütüphane bir soket sınıfı verir. Nesnede karşı tarafın IP adresi ve portu yazılır.
- Yapı:
  - Frontier manager tohum URL'leri listeye koyar ve tüm iş parçacıklarını başlatır. Bu modül paralelleştirilmez.
  - Her iş parçacığı bir sıralı tarayıcı gibi çalışır: URL al, indir, URL çıkar, kaydet, durdurma koşulunu kontrol et.
  - Hepsi aynı sınır listesini ve aynı depoyu paylaşır. İkisine erişim senkronize edilir.
- DNS sorgusu ve depoya yazma (G/Ç erişimi) hâlâ yavaştır. Bunlara ayrı çözüm gerekir.

Evrensel tarayıcı (universal crawler)

- Genel arama motorlarını destekler. Milyarlarca sayfaya ölçeklenmeli.
- Hoca bunların ağ bant genişliğinin %50'sinden fazlasını kullandığını söyler.
- Dizini her seferinde sıfırdan kurmaz. Yeni sayfaları ekleyerek dizini artımlı (incremental) günceller.
- İki önemli etken: performans ve politika.
- Politika: kapsam ile güncellik dengesi.
  - Kapsam: olabildiğince çok farklı sayfayı dizinlemek.
  - Güncellik: dizindeki sayfaları sık ziyaret edip güncel tutmak.
- Mimari iyileştirmeler:

  - DNS için önbellek.
  - Eşzamansız soket: yanıt beklenmeden aynı soketten yeni istek gönderilir. Makinede açılabilecek soket sayısı artar.
  - Yazılım tabanlı ağ anahtarları: yönlendirme (routing) süresi kısalır.
  - Yazılım tabanlı depolama anahtarları: G/Ç erişimi hızlanır.
- Arama motorlarının dizin boyutları farklıdır ve yıldan yıla değişir. Dizin boyutu web'deki veri miktarı hakkında fikir verir.
- Güncellik için sayfadaki `Last-Modified` ve `Expires` bilgilerine bakılır (altyazıda "Glass Modified" diye bozuk). Bu bilgiyle sayfanın ne zaman yeniden ziyaret edileceği belirlenir.

Tercihli tarayıcılar (preferential crawlers)

- URL'ye önem ölçüsü (importance measure) verilir. Sınır listesi bu değere göre sıralanır.
- Önem değeri nasıl hesaplanır:
  - O ana kadar taranan sayfalardan bir web çizgesi kurulur. Bağlantı tabanlı sıralama algoritması (ör. PageRank) çalıştırılır.
  - URL'deki kelimeler ile konu kelimeleri arasındaki benzerliğe bakılır.
  - URL'nin bulunduğu üst (parent) sayfadaki kelimeler ile konu arasındaki benzerliğe bakılır.
- Focused tarayıcı:
  - Gözetimli (supervised) çalışır: bir sınıflandırıcı (classifier) kullanır.
  - Ör. yemek tarifi tarayıcısı indirdiği sayfayı "tarif" veya "tarif değil" diye etiketler.
  - Tarif ise sayfadaki URL'ler sınır listesine eklenir. Değilse eklenmez.
- Topical tarayıcı:
  - Bağlantı metni (anchor text) ve üst sayfadaki kelimelerin konuya benzerliğine bakar.
  - Benzerlik eşiğin üstündeyse URL listeye eklenir.
- Algoritmalar:
  - BFS ve DFS tercihli değildir. Kıyaslama (benchmark) için kullanılır.
  - Best-First (altyazıda "best and first", büyük olasılıkla Best-N-First): konuya benzer sayfalardan gelen URL'lere yüksek değer verilir. Liste benzerliğe göre sıralıdır. En üstteki URL'ler paralel indirilir.
  - PageRank tabanlı: URL'ler anahtar kelimelere ve PageRank değerine göre sıralanır.
  - Fish Search ve onun geliştirilmiş hâli Shark Search: bağlantı metnindeki ve üst sayfadaki kelimelere bakar.
  - InfoSpider: yeni sayfalardan çıkan URL'lerle arar. Topical tarayıcı örneğidir.

Karşılaştırma: crawl size–recall

- Yatay eksen: taranan sayfa oranı (%0–%100). Dikey eksen: duyarlılık (recall).
- Duyarlılık: bulunan ilgili sayfaların tüm ilgili sayfalara oranı.
- %100'de tüm algoritmalar tüm ilgili sayfaları bulur. Önemli olan düşük tarama boyutunda (%20–%40) hangi algoritmanın daha çok ilgili sayfa bulduğu.
- Hocanın gösterdiği grafikte karşılaştırılanlar: PageRank, BFS, rastgele (random), geri bağlantı (backlink) tabanlı.
- Grafikte PageRank tabanlı algoritma düşük tarama boyutunda bile en çok ilgili sayfayı buluyor.

Topical tarayıcı örneği

- Girdi: bir konu (sorgu, açıklama veya anahtar kelime) ve tohum sayfa kümesi.
- Dizin kurmaz. Arama başlayınca taramaya başlar.
- InfoSpider örneğinde bulunan sonuçlara kelime grubuna yakınlığa göre puan verilir. Sonuçlar yukarıdan aşağıya sıralı gösterilir. Yeni sonuç bulununca listede yerine yerleşir.

Kaynaklar ve duyurular

- Slaytta açık kaynak tarayıcı bağlantıları var. Hocanın en çok kullandığı tarayıcı altyazıda "notch" diye geçer (büyük olasılıkla Apache Nutch). Kendi arama motorunu da içerir.
- Bu araçlar olduğu gibi kullanılmaz. Projede üzerine kod yazmak gerekir.

- Gelecek hafta: bağlantı analizi (link analysis), kitabın 7. bölümü. Slaytlar dersin sayfasında.
- Proje sorusu olan derste sorabilir veya e-posta atabilir.

Ders sonrası notlar

- Hoca sınır listesine ekleme adımını anlatırken (00:56:38) “son giren ilk çıkar” listede yeni URL’lerin sona eklendiğini söyler. Sona ekleyip baştan çekmek büyük olasılıkla FIFO’dur. Doğrusu: BFS için FIFO kuyruk, DFS için LIFO yığın kullanılır. Hoca devamında bunu doğru biçimde anlatır.
- “Glass Modified” altyazı hatasıdır. Doğru HTTP başlığı: `Last-Modified`.
- “Birthday DB” büyük olasılıkla Berkeley DB, “notch crawler” büyük olasılıkla Apache Nutch. İkisi de altyazıdan çıkarılmıştır.
- Hoca örnek kodu Perl ile yazar. Slayttaki kod büyük olasılıkla ders kitabındaki örnektir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.